시뮬레이션 테스트 분석 문서

개요

이 문서는 Mars 블록체인 프로젝트의 시뮬레이션 테스트 시스템을 상세히 분석합니다. 시뮬레이션 테스트는 Cosmos SDK의 핵심 기능 중 하나로, 랜덤 트랜잭션을 생성하여 블록체인의 장기 안정성과 결정론적 동작을 검증합니다.

시뮬레이션 테스트 아키텍처

1. 전체 구조

```
시뮬레이션 테스트 시스템
├── app/sim_test.go # 메인 시뮬레이션 테스트
├── app/sim_bench_test.go # 성능 벤치마크 테스트
└── x/mars/module/simulation.go # Mars 모듈 시뮬레이션 구현
```

2. 핵심 컴포넌트

시뮬레이션 설정 (SimulationConfig)

- **ChainID**: 시뮬레이션 체인 식별자 (“mars-simapp”)
- **NumBlocks**: 시뮬레이션할 블록 수
- **BlockSize**: 각 블록의 트랜잭션 수
- **Seed**: 랜덤 시드 값 (재현 가능한 테스트를 위함)

- **Commit**: 상태 커밋 여부

시뮬레이션 환경

- **Database**: LevelDB 기반 임시 데이터베이스
- **Logger**: 테스트 로거 (상세 로그 옵션 지원)
- **AppOptions**: 애플리케이션 설정 옵션

시뮬레이션 테스트 유형

1. BenchmarkSimulation (성능 벤치마크)

```
// 벤치마크 시뮬레이션 실행

// Ignite CLI 명령어: ignite chain simulate -v --numBlocks 200 --blockSize 50

// Go 벤치마크: go test -benchmem -run=^$ -bench ^BenchmarkSimulation ./app -NumB

func BenchmarkSimulation(b *testing.B) {

// 시뮬레이션 설정 초기화

simcli.FlagSeedValue = time.Now().Unix() // 현재 시간을 시드로 사용

simcli.FlagVerboseValue = true  // 상세 로그 활성화

simcli.FlagCommitValue = true  // 상태 커밋 활성화

simcli.FlagEnabledValue = true  // 시뮬레이션 활성화

// 설정 객체 생성 및 체인 ID 설정

config := simcli.NewConfigFromFlags()

config.ChainID = SimAppChainID

// 시뮬레이션 환경 설정 (데이터베이스, 디렉토리, 로거)

db, dir, logger, skip, err := simtestutil.SetupSimulation(
```

```
config, "leveldb-app-sim", "Simulation",
simcli.FlagVerboseValue, simcli.FlagEnabledValue
)
// 애플리케이션 인스턴스 생성
bApp := New(logger, db, nil, true, appOptions,
fauxMerkleModeOpt, // 빠른 시뮬레이션을 위한 가짜 머클 모드
baseapp.SetChainID(SimAppChainID) // 체인 ID 설정
)
// 랜덤 시뮬레이션 실행
_, simParams, simErr := simulation.SimulateFromSeed(
b, // 테스트 컨텍스트
os.Stdout, // 출력 스트림
bApp.BaseApp, // 베이스 애플리케이션
simtestutil.AppStateFn(...), // 앱 상태 생성 함수
simulationtypes.RandomAccounts, // 랜덤 계정 생성 함수
simtestutil.SimulationOperations(...), // 시뮬레이션 작업 목록
BlockedAddresses(), // 차단된 주소 목록
config, // 시뮬레이션 설정
bApp.AppCodec(), // 코덱
)
}
```

특징:

- 성능 측정에 최적화
- 메모리 할당 추적 (`b.ReportAllocs()`)
- CPU 프로파일링 지원
- 빠른 실행을 위한 `fauxMerkleModeOpt` 사용

2. TestFullAppSimulation (전체 애플리케이션 시뮬레이션)

```
func TestFullAppSimulation(t *testing.T) {
// 시뮬레이션 설정 및 환경 구성
config := simcli.NewConfigFromFlags()
config.ChainID = SimAppChainID
// 애플리케이션 생성 및 서명 검증 설정
app := New(logger, db, nil, true, appOptions, fauxMerkleModeOpt, baseapp.SetChai
if !simcli.FlagSigverifyTxValue {
app.SetNotSigverifyTx() // 시뮬레이션에서 서명 검증 비활성화
}
// 랜덤 시뮬레이션 실행 및 결과 검증
_, simParams, simErr := simulation.SimulateFromSeed(...)
// 상태 내보내기 검증
err = simtestutil.CheckExportSimulation(app, config, simParams)
require.NoError(t, err)
require.NoError(t, simErr)
}
```

특징:

- 전체 애플리케이션 기능 테스트
- 상태 내보내기/가져오기 검증
- 서명 검증 옵션 제어

3. TestAppImportExport (상태 가져오기/내보내기 테스트)

```
func TestAppImportExport(t *testing.T) {
// 첫 번째 앱에서 시뮬레이션 실행
bApp := New(logger, db, nil, true, appOptions, fauxMerkleModeOpt, baseapp.SetCha
_, simParams, simErr := simulation.SimulateFromSeed(...)
// 제네시스 상태 내보내기
fmt.Printf("exporting genesis...\n")
exported, err := bApp.ExportAppStateAndValidators(false, []string{}, []string{})
require.NoError(t, err)
// 새로운 앱 인스턴스 생성
fmt.Printf("importing genesis...\n")
newApp := New(log.NewNopLogger(), newDB, nil, true, appOptions, fauxMerkleModeOp
// 내보낸 상태를 새 앱에 가져오기
var genesisState GenesisState
err = json.Unmarshal(exported.AppState, &genesisState)
require.NoError(t, err)
// 제네시스 초기화 및 상태 비교
_, err = newApp.ModuleManager.InitGenesis(ctxB, bApp.AppCodec(), genesisState)
```

```go
// 저장소 상태 비교
for _, appKeyA := range storeKeys {
storeA := ctxA.KVStore(appKeyA)
storeB := ctxB.KVStore(appKeyB)
failedKVAs, failedKVBs := simtestutil.DiffKVStores(storeA, storeB, skipPrefixes[
require.Equal(t, len(failedKVAs), len(failedKVBs), "unequal sets of key-values t
require.Equal(t, 0, len(failedKVAs), simtestutil.GetSimulationLog(...))
}
}
```

특징:

- 상태 직렬화/역직렬화 검증
- 두 앱 인스턴스 간 상태 일관성 확인
- 특정 저장소 키 제외 기능 (`skipPrefixes`)

4. TestAppSimulationAfterImport (가져오기 후 시뮬레이션)

```go
func TestAppSimulationAfterImport(t *testing.T) {
// 첫 번째 시뮬레이션 실행
stopEarly, simParams, simErr := simulation.SimulateFromSeed(...)
// 조기 종료 확인 (검증자가 없는 경우)
if stopEarly {
fmt.Println("can't export or import a zero-validator genesis, exiting test...")
```

```go
return
}
// 상태 내보내기 및 새 앱에서 가져오기
exported, err := bApp.ExportAppStateAndValidators(true, []string{}, []string{})
newApp := New(...)
// 체인 초기화
_, err = newApp.InitChain(&abci.RequestInitChain{
AppStateBytes: exported.AppState,
ChainId: SimAppChainID,
})
// 가져온 상태에서 추가 시뮬레이션 실행
_, _, err = simulation.SimulateFromSeed(...)
require.NoError(t, err)
}
```

특징:

- 가져온 상태에서의 연속 시뮬레이션
- 검증자 상태 확인
- ABCI 초기화 프로세스 테스트

5. TestAppStateDeterminism (결정론적 동작 검증)

```go
func TestAppStateDeterminism(t *testing.T) {
// 시뮬레이션 설정
```

```go
config := simcli.NewConfigFromFlags()

config.InitialBlockHeight = 1

config.ExportParamsPath = ""

config.OnOperation = true  // 각 작업마다 불변성 검사

config.AllInvariants = true  // 모든 불변성 검사 활성화

numSeeds := 3  // 테스트할 시드 수

numTimesToRunPerSeed := 3  // 각 시드당 실행 횟수

appHashList := make([]json.RawMessage, numTimesToRunPerSeed)

// 여러 시드에 대해 반복 테스트

for i := 0; i < numSeeds; i++ {

if config.Seed == simcli.DefaultSeedValue {

config.Seed = rand.Int63() // 랜덤 시드 생성

}

// 동일 시드로 여러 번 실행

for j := 0; j < numTimesToRunPerSeed; j++ {

bApp := New(logger, db, nil, true, appOptions,

interBlockCacheOpt(), // 블록 간 캐시 활성화

baseapp.SetChainID(SimAppChainID)

)

// 시뮬레이션 실행

_, _, err := simulation.SimulateFromSeed(...)

require.NoError(t, err)

// 앱 해시 비교
```

```
appHash := bApp.LastCommitID().Hash

appHashList[j] = appHash

if j != 0 {

// 동일 시드로 실행한 결과는 반드시 같아야 함

require.Equal(t, string(appHashList[0]), string(appHashList[j]),

"non-determinism in seed %d: %d/%d, attempt: %d/%d\n",

config.Seed, i+1, numSeeds, j+1, numTimesToRunPerSeed,

)

}

}

}

}
```

특징:

- 결정론적 실행 보장 검증
- 동일 시드에서 동일 결과 확인
- 불변성 검사 활성화
- 앱 해시 비교를 통한 상태 일관성 확인

시뮬레이션 파라미터 및 설정

1. 핵심 플래그 및 설정

```
// 시뮬레이션 활성화 플래그
```

```go
var FlagEnableStreamingValue bool


func init() {
simcli.GetSimulatorFlags() // 시뮬레이션 플래그 초기화
flag.BoolVar(&FlagEnableStreamingValue, "EnableStreaming", false, "Enable stream
}


// 주요 시뮬레이션 설정
type SimulationConfig struct {
Seed int64  // 랜덤 시드
NumBlocks int  // 시뮬레이션할 블록 수
BlockSize int  // 블록당 트랜잭션 수
Commit bool  // 상태 커밋 여부
Verbose bool  // 상세 로그 출력
Enabled bool  // 시뮬레이션 활성화
InitialBlockHeight int  // 시작 블록 높이
ExportParamsPath string  // 파라미터 내보내기 경로
OnOperation bool  // 작업별 불변성 검사
AllInvariants bool  // 모든 불변성 검사
}
```

2. 성능 최적화 옵션

```go
// 가짜 머클 모드 - 시뮬레이션 속도 향상
func fauxMerkleModeOpt(bapp *baseapp.BaseApp) {
bapp.SetFauxMerkleMode() // IAVL 대신 dbStoreAdapter 사용
}

// 블록 간 캐시 - 성능 향상
func interBlockCacheOpt() func(*baseapp.BaseApp) {
return baseapp.SetInterBlockCache(store.NewCommitKVStoreCacheManager())
}

// 스트리밍 서비스 설정
if FlagEnableStreamingValue {
m := make(map[string]interface{})
m["streaming.abci.keys"] = []string{"*"} // 모든 키 스트리밍
m["streaming.abci.plugin"] = "abci_v1"  // ABCI v1 플러그인
m["streaming.abci.stop-node-on-err"] = true  // 에러 시 노드 중지
}
```

3. 데이터베이스 및 저장소 설정

```go
// 시뮬레이션용 임시 데이터베이스 설정
db, dir, logger, skip, err := simtestutil.SetupSimulation(
config, // 시뮬레이션 설정
```

```
"leveldb-app-sim", // 데이터베이스 이름
"Simulation", // 시뮬레이션 이름
simcli.FlagVerboseValue, // 상세 로그 여부
simcli.FlagEnabledValue // 시뮬레이션 활성화 여부
)

// 메모리 데이터베이스 (결정론 테스트용)
db := dbm.NewMemDB()
```

Mars 모듈 시뮬레이션 구현

1. 제네시스 상태 생성

```
// GenerateGenesisState - 랜덤 제네시스 상태 생성
func (AppModule) GenerateGenesisState(simState *module.SimulationState) {
// 시뮬레이션 계정 주소 추출
accs := make([]string, len(simState.Accounts))
for i, acc := range simState.Accounts {
accs[i] = acc.Address.String()
}
// Mars 모듈 제네시스 상태 생성
marsGenesis := types.GenesisState{
Params: types.DefaultParams(), // 기본 파라미터 사용
```

```
}

// 제네시스 상태를 시뮬레이션 상태에 추가

simState.GenState[types.ModuleName] = simState.Cdc.MustMarshalJSON(&marsGenesis)

}
```

2. 저장소 디코더 등록

```
// RegisterStoreDecoder - 저장소 디코더 등록

func (am AppModule) RegisterStoreDecoder(_ simtypes.StoreDecoderRegistry) {

// Mars 모듈은 단순한 구조로 별도 디코더 불필요

// 향후 복잡한 데이터 구조 추가 시 디코더 등록 가능

}
```

3. 가중치 기반 작업 정의

```
// WeightedOperations - 시뮬레이션 작업 목록 반환

func (am AppModule) WeightedOperations(simState module.SimulationState) []simtyp

operations := make([]simtypes.WeightedOperation, 0)

// 현재는 빈 목록 반환 (Mars 모듈에 시뮬레이션 가능한 트랜잭션 없음)

// 향후 추가 가능한 작업 예시:

// operations = append(operations, simtypes.WeightedOperation{

// Weight: 100, // 실행 가중치

// Op: SimulateMsgUpdateParams(...), // 시뮬레이션 함수
```

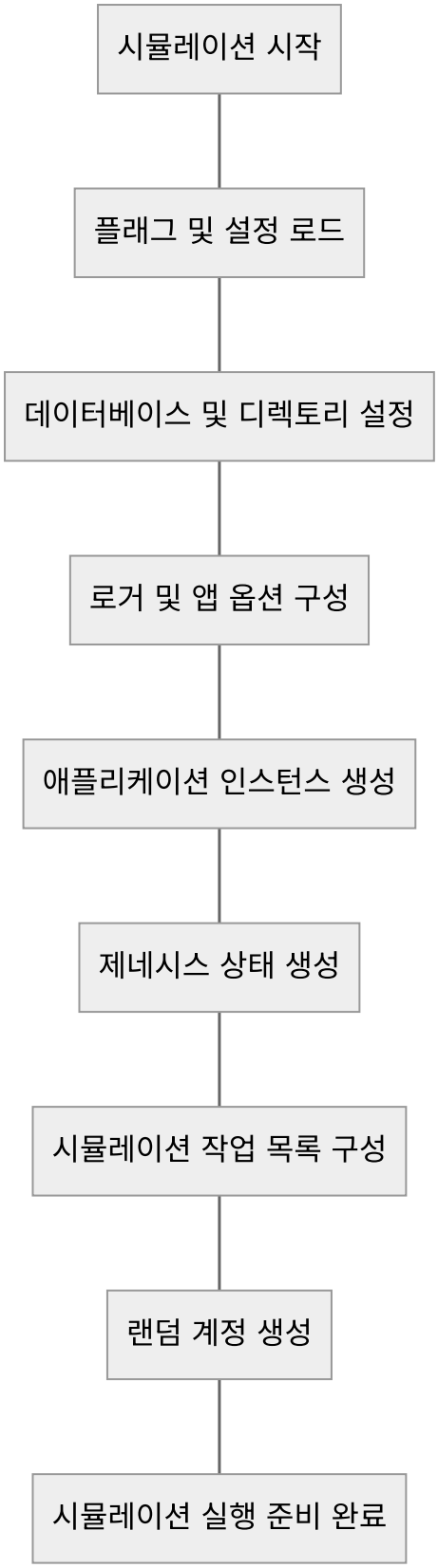
```
// })
return operations
}
```

4. 거버넌스 제안 시뮬레이션

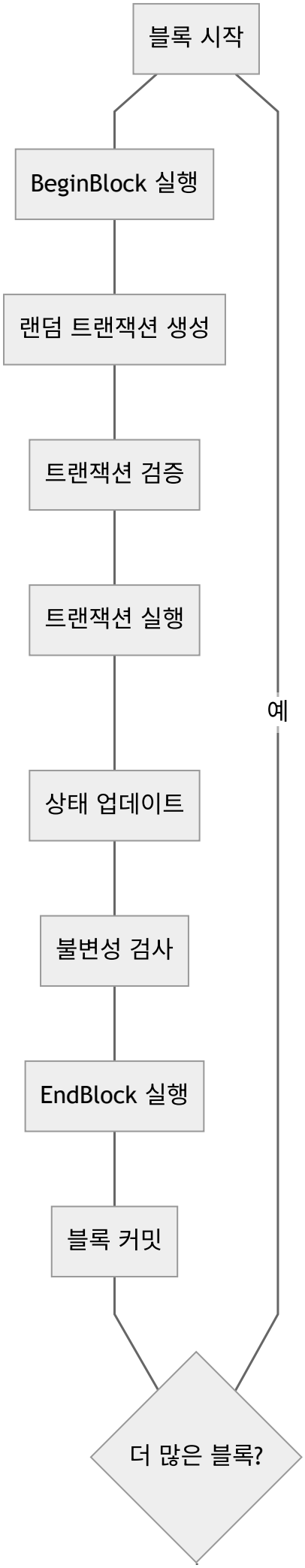
```
// ProposalMsgs - 거버넌스 제안 시뮬레이션 메시지
func (am AppModule) ProposalMsgs(simState module.SimulationState) []simtypes.Wei
// 현재는 빈 목록 반환
// 향후 거버넌스 기반 파라미터 업데이트 제안 시뮬레이션 추가 가능
return []simtypes.WeightedProposalMsg{}
}
```

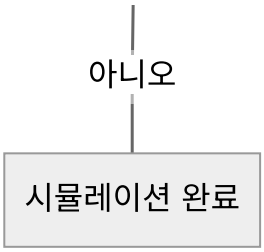
시뮬레이션 실행 과정

1. 시뮬레이션 초기화 단계

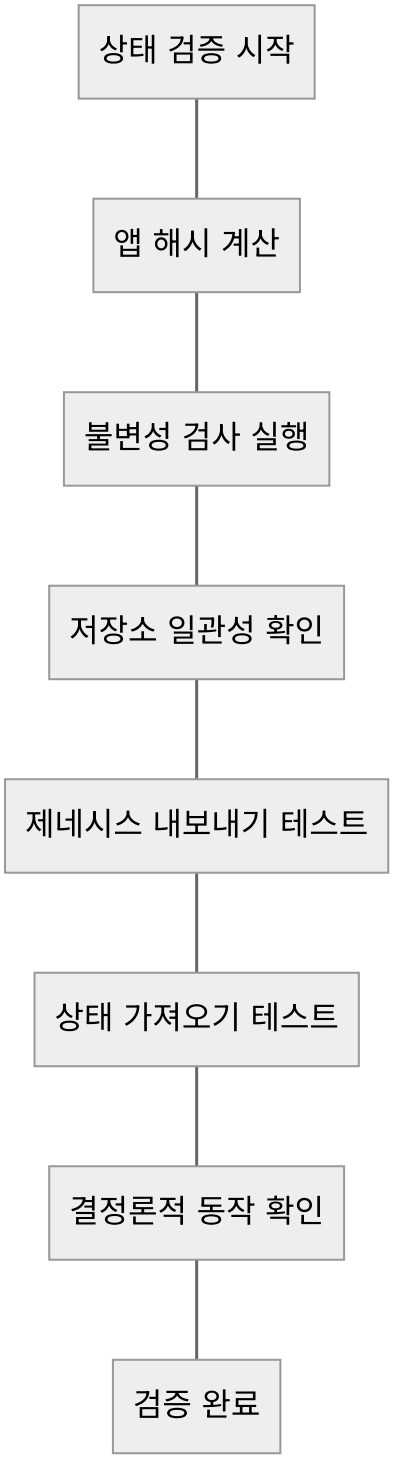


2. 블록 생성 및 트랜잭션 처리


블록 시작
BeginBlock 실행
랜덤 트랜잭션 생성
트랜잭션 검증
트랜잭션 실행
상태 업데이트
불변성 검사
EndBlock 실행
블록 커밋
더 많은 블록?
예



3. 상태 검증 과정



장기 안정성 테스트 방법론

1. 다중 시드 테스트

```
// 여러 랜덤 시드로 반복 테스트
numSeeds := 3
for i := 0; i < numSeeds; i++ {
if config.Seed == simcli.DefaultSeedValue {
config.Seed = rand.Int63() // 새로운 랜덤 시드 생성
}
// 동일 시드로 여러 번 실행하여 결정론적 동작 확인
for j := 0; j < numTimesToRunPerSeed; j++ {
// 시뮬레이션 실행 및 결과 비교
}
}
```

목적:

- 다양한 시나리오에서의 안정성 확인
- 결정론적 동작 보장
- 예외 상황 처리 검증

2. 장기간 블록 생성 테스트

```
# Ignite CLI를 통한 장기 시뮬레이션
ignite chain simulate -v --numBlocks 10000 --blockSize 100
```

```
# Go 테스트를 통한 장기 시뮬레이션
go test -run TestFullAppSimulation ./app -NumBlocks=10000 -BlockSize=100 -
```

검증 항목:

- 메모리 누수 없음
- 성능 저하 없음
- 상태 일관성 유지
- 불변성 조건 만족

3. 스트레스 테스트

```
// 높은 트랜잭션 부하 테스트
config.NumBlocks = 1000
config.BlockSize = 200  // 블록당 200개 트랜잭션

// 메모리 사용량 모니터링
func BenchmarkFullAppSimulation(b *testing.B) {
b.ReportAllocs() // 메모리 할당 추적
// CPU 프로파일링
// go test -cpuprofile cpu.out -bench BenchmarkFullAppSimulation
}
```

4. 상태 무결성 검증

```
// 불변성 검사 활성화
config.OnOperation = true  // 각 작업마다 불변성 검사
config.AllInvariants = true  // 모든 불변성 검사 활성화


// 저장소 상태 비교
failedKVAs, failedKVBs := simtestutil.DiffKVStores(storeA, storeB, skipPrefixes[
require.Equal(t, 0, len(failedKVAs), "상태 불일치 발견")
```

5. 복구 및 재시작 테스트

```
// 상태 내보내기 후 새 인스턴스에서 가져오기
exported, err := bApp.ExportAppStateAndValidators(false, []string{}, []string{})
newApp := New(...)
_, err = newApp.ModuleManager.InitGenesis(ctxB, bApp.AppCodec(), genesisState)


// 가져온 상태에서 추가 시뮬레이션 실행
_, _, err = simulation.SimulateFromSeed(...)
```

성능 최적화 및 모니터링

1. 성능 최적화 기법

```go
// 1. 가짜 머클 모드 사용 (시뮬레이션 전용)
func fauxMerkleModeOpt(bapp *baseapp.BaseApp) {
bapp.SetFauxMerkleMode() // IAVL 대신 빠른 저장소 사용
}

// 2. 블록 간 캐시 활성화
func interBlockCacheOpt() func(*baseapp.BaseApp) {
return baseapp.SetInterBlockCache(store.NewCommitKVStoreCacheManager())
}

// 3. 서명 검증 비활성화 (시뮬레이션에서만)
if !simcli.FlagSigverifyTxValue {
app.SetNotSigverifyTx()
}
```

2. 메모리 사용량 모니터링

```go
// 벤치마크에서 메모리 할당 추적
func BenchmarkFullAppSimulation(b *testing.B) {
b.ReportAllocs() // 메모리 할당 통계 출력
// 시뮬레이션 실행
_, simParams, simErr := simulation.SimulateFromSeed(...)
// 데이터베이스 통계 출력
```

```
if config.Commit {
simtestutil.PrintStats(db)
}
}
```

3. CPU 프로파일링

```
# CPU 프로파일링과 함께 벤치마크 실행
go test -benchmem -run=^$ ./app -bench ^BenchmarkFullAppSimulation$ \
-Commit=true -cpuprofile cpu.out

# 프로파일 분석
go tool pprof cpu.out
```

시뮬레이션 결과 분석

1. 통계 정보

```
// 데이터베이스 통계 출력
if config.Commit {
simtestutil.PrintStats(db)
}
```

```
// 출력 예시:
// Database stats:
// - Total keys: 1234
// - Total size: 567KB
// - Average key size: 32 bytes
// - Average value size: 128 bytes
```

2. 시뮬레이션 파라미터 검증

```
// 시뮬레이션 파라미터 내보내기 및 검증
err = simtestutil.CheckExportSimulation(app, config, simParams)
require.NoError(t, err)

// 파라미터 내용:
// - 실행된 작업 수
// - 성공/실패 통계
// - 평균 실행 시간
// - 가스 사용량 통계
```

3. 에러 및 예외 처리

```
// 시뮬레이션 에러 처리
_, simParams, simErr := simulation.SimulateFromSeed(...)
```

```go
if simErr != nil {
// 에러 로그 출력
fmt.Printf("Simulation error: %v\n", simErr)
// 스택 트레이스 출력
fmt.Printf("Stack trace: %s\n", string(debug.Stack()))
}

// 조기 종료 처리 (검증자 없음)
if stopEarly {
fmt.Println("can't export or import a zero-validator genesis, exiting test...")
return
}
```

시뮬레이션 테스트 실행 방법

1. Ignite CLI를 통한 실행

```bash
# 기본 시뮬레이션
ignite chain simulate

# 상세 로그와 함께 실행
ignite chain simulate -v
```

```
# 커스텀 파라미터로 실행
ignite chain simulate -v --numBlocks 1000 --blockSize 50


# 특정 시드로 실행 (재현 가능한 테스트)
ignite chain simulate --seed 12345
```

2. Go 테스트를 통한 실행

```
# 전체 시뮬레이션 테스트
go test -run TestFullAppSimulation ./app


# 벤치마크 테스트
go test -bench BenchmarkSimulation ./app


# 커스텀 파라미터로 테스트
go test -run TestFullAppSimulation ./app \
-NumBlocks=500 -BlockSize=25 -Commit=true -Verbose=true


# 결정론 테스트
go test -run TestAppStateDeterminism ./app


# 상태 가져오기/내보내기 테스트
go test -run TestAppImportExport ./app
```

3. 성능 프로파일링

```
# CPU 프로파일링
go test -bench BenchmarkFullAppSimulation ./app -cpuprofile cpu.out

# 메모리 프로파일링
go test -bench BenchmarkFullAppSimulation ./app -memprofile mem.out

# 프로파일 분석
go tool pprof cpu.out
go tool pprof mem.out
```

결론

Mars 블록체인의 시뮬레이션 테스트 시스템은 다음과 같은 핵심 기능을 제공합니다:

1. **랜덤 트랜잭션 생성**: 다양한 시나리오에서의 블록체인 동작 검증
2. **결정론적 동작 확인**: 동일 입력에 대한 동일 출력 보장
3. **장기 안정성 검증**: 수천 개 블록에 걸친 안정성 테스트
4. **상태 무결성 검사**: 불변성 조건 및 저장소 일관성 확인
5. **성능 모니터링**: 메모리 사용량 및 CPU 성능 추적

이러한 시뮬레이션 테스트를 통해 Mars 블록체인의 안정성과 신뢰성을 보장하며, 프로덕션 환경에서 발생할 수 있는 다양한 상황에 대한 대비책을 마련할 수 있습니다.